Firebase Authentication and Bearer Token Sequence Diagram

Project: RoomRadar QC

Purpose

This document explains how RoomRadar QC uses Firebase Authentication and Firebase Admin SDK to protect the admin CSV upload feature. It also includes a sequence diagram showing how the frontend, Firebase, backend, and database interact during admin login and semester data upload.

This document addresses the request for a UML-style sequence diagram explaining the authentication and bearer token flow used in the project.

---

1. Why Firebase Was Used

RoomRadar QC uses Firebase Authentication for admin login and Firebase Admin SDK for backend token verification.

Firebase was used because it provides a managed authentication system without requiring the team to build a complete custom password system from scratch. This allowed the team to focus on the main project functionality: searching for room availability based on schedule data and allowing admins to upload semester CSV files.

The main reason Firebase was added was to protect the admin upload workflow. Only an authenticated admin user should be able to upload a semester schedule CSV file.

---

2. What Firebase Does in This Project

Firebase is used for two main purposes:

Firebase Component	Purpose in RoomRadar QC
Firebase Authentication	Allows an admin user to log in with email and password from the React frontend.
Firebase ID Token	Acts as proof that the user successfully logged in.
Firebase Admin SDK	Allows the Flask backend to verify that the ID token is valid.

The frontend handles login. The backend handles verification.

This means the frontend does not simply tell the backend, “this user is an admin.” Instead, the frontend sends a Firebase ID token, and the backend verifies that token before allowing the CSV upload.

---

3. Admin Authentication Flow Summary

The admin authentication process works like this:

1. Admin enters email and password on the Admin Login page.
2. React frontend sends the login request to Firebase Authentication.
3. Firebase Authentication verifies the email and password.
4. Firebase returns an authenticated user.
5. The React frontend requests an ID token from Firebase.
6. The frontend stores the token in local storage as adminToken.
7. When the admin uploads a CSV file, the frontend sends the token in the Authorization header.
8. The Flask backend reads the Authorization header.
9. The Flask backend verifies the token using Firebase Admin SDK.
10. If the token is valid, the backend processes the CSV file and inserts semester schedule data into MySQL.
11. If the token is missing or invalid, the backend rejects the request.

---

4. Bearer Token Explanation

RoomRadar QC uses the Firebase ID token as a bearer token.

A bearer token means that whoever sends a valid token is treated as an authenticated user for that request. The token is sent in the HTTP Authorization header.

Example:

Authorization: Bearer <firebase_id_token>

The backend does not trust the token blindly. It verifies the token using Firebase Admin SDK before allowing the upload.

In the backend upload route, the token is checked before the CSV file is processed.

If the token is missing, the backend returns:

401 Unauthorized

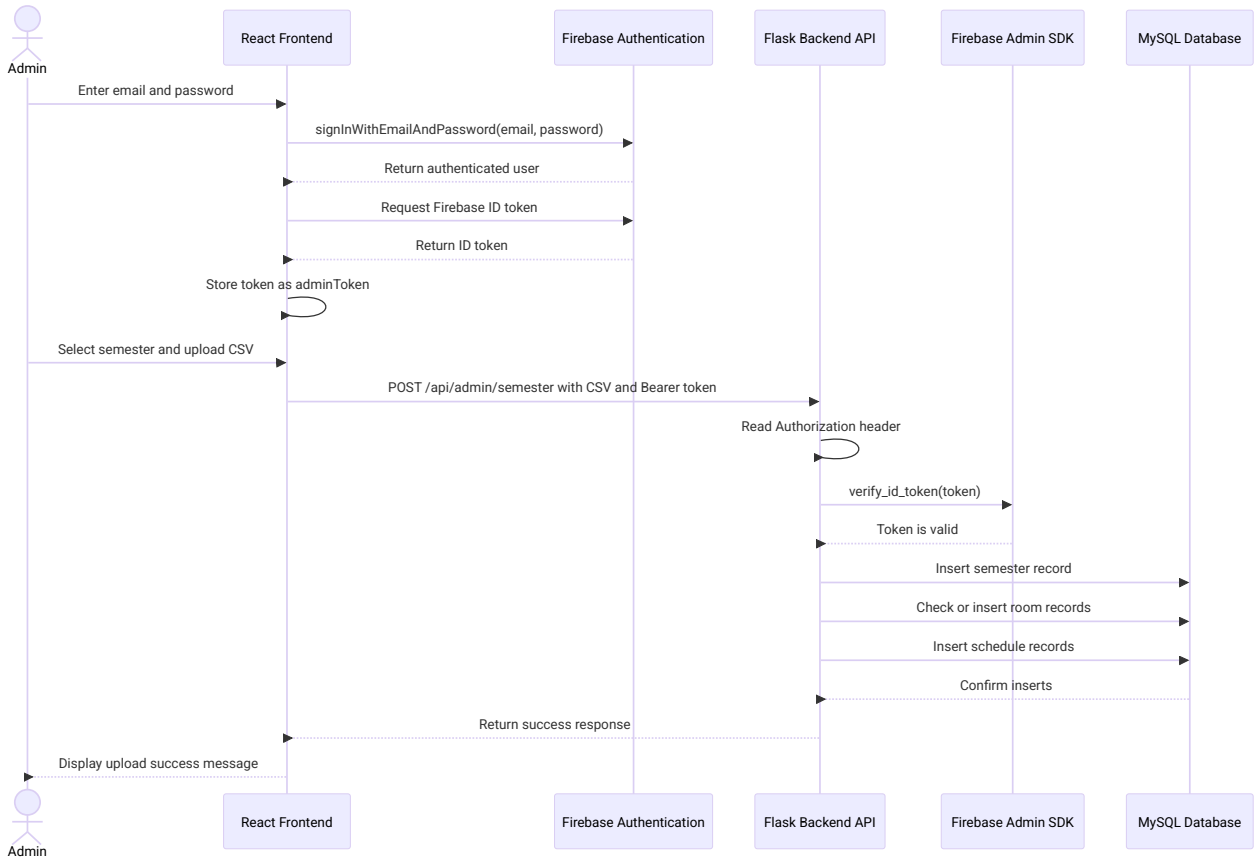
If the token is invalid, the backend also returns:

401 Unauthorized

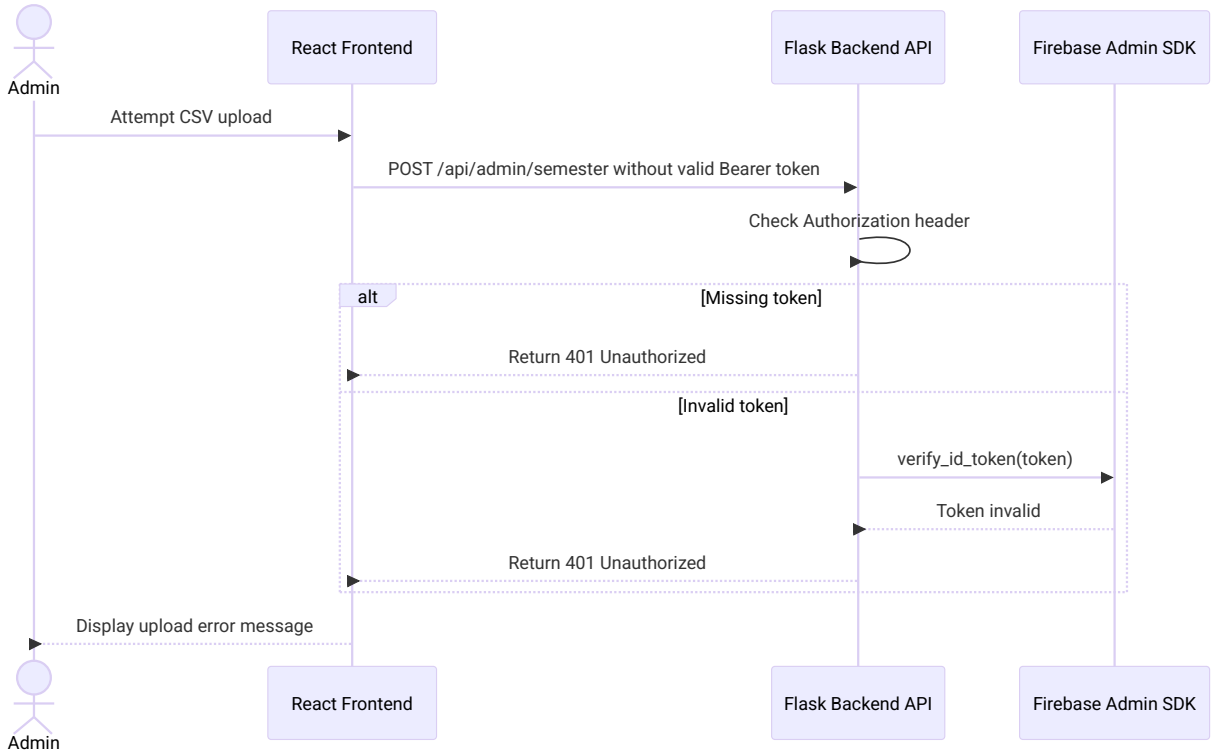
Only after the token is verified does the backend continue with the semester upload process.

---

5. Sequence Diagram: Admin Login and CSV Upload



6. Sequence Diagram: Missing or Invalid Token



7. Backend Authentication Logic

The backend upload route checks the request before processing the CSV file.

The backend expects an Authorization header:

Authorization: Bearer <firebase_id_token>

The backend then removes the Bearer part and verifies the token.

General backend logic:

```
auth_header = request.headers.get("Authorization")

if not auth_header:
    return jsonify({"message": "Unauthorized"}), 401

token = auth_header.replace("Bearer ", "")

try:
    auth.verify_id_token(token)
except:
    return jsonify({"message": "Invalid token"}), 401
```

Only after this check passes does the backend continue with:

- Reading the semester name
- Reading the uploaded CSV file
- Creating a semester record
- Inserting room records
- Inserting schedule records

---

8. Responsibility Breakdown

Area	Responsible Team Member(s)	Explanation

Admin login frontend	Kyrilous	Built the Admin Login page and handled the frontend login flow.
Token storage and frontend request header	Kyrilous	Stored the Firebase ID token and sent it in the Authorization header during CSV upload.
Backend token verification	Darshan	Implemented or supported backend verification using Firebase Admin SDK.
Admin upload backend route	Darshan	Worked on the Flask route that processes semester CSV uploads after token verification.
Admin workflow testing	Joseph	Reviewed and tested admin access, protected workflow behavior, and upload behavior.
Documentation and explanation	Kyrilous, Darshan, Joseph	Team contributed to explaining and reviewing the authentication design.

---

9. Why Not Store Admin Accounts Directly in MySQL?

An alternative design would be to store admin accounts directly in the MySQL database and authenticate admins through the Flask backend.

That approach could work, but it would require the team to build more custom authentication features, such as:

- User password storage
- Password hashing
- Login route
- Session or token generation
- Password reset handling
- Admin role checks
- More security testing for custom authentication

Firebase reduced the amount of custom authentication logic the team needed to build. Instead of manually managing passwords, Firebase handled login and token generation. The Flask backend only needed to verify the token before allowing protected actions.

10. Tradeoff of Using Firebase

Advantages

Advantage	Explanation
Managed authentication	Firebase handles email/password authentication.
Token-based access	The frontend receives an ID token that can be verified by the backend.
Reduced password management	The team does not need to manually store or hash passwords.
Backend verification	Flask can verify tokens using Firebase Admin SDK.

Disadvantages

Disadvantage	Explanation
Additional service	Firebase adds another external service to the architecture.
Configuration needed	Firebase credentials must be configured securely in local and deployed environments.
Admin role limitation	The current implementation verifies that the user is authenticated, but it can be improved with stricter admin role checks.

---

11. Current Security Limitation

The current backend authentication check verifies that the uploaded request includes a valid Firebase ID token.

This confirms that the user is authenticated through Firebase.

However, the current version can be improved by also checking whether the authenticated

Firebase user is specifically an approved admin user.

For example, a future version could check:

- Firebase user email
- Firebase UID
- Firebase custom claims
- A MySQL admins table containing approved admin users

This would make the system stricter by verifying both:

User is logged in
AND
User is approved as an admin

---

12. Future Improvement: Admin Role Authorization

A future improvement would be to add role-based authorization.

One possible design:

1. Admin logs in through Firebase.
2. Frontend receives Firebase ID token.
3. Frontend sends token to backend.
4. Backend verifies token with Firebase Admin SDK.
5. Backend extracts the user email or UID from the verified token.
6. Backend checks the email or UID against an approved admin list.
7. Backend only allows CSV upload if the user is approved.

\